深度之眼
deepshare.net

神经网络基石：从零构建你的第一个模型

导师：择木老师

深度之眼
deepshare.net

目录

1 MLP架构和激活函数

2 损失函数与网络训练

3 梯度下降与反向传播

4 Pytorch回归任务实例

深度之眼
deepshare.net

MLP架构和激活函数

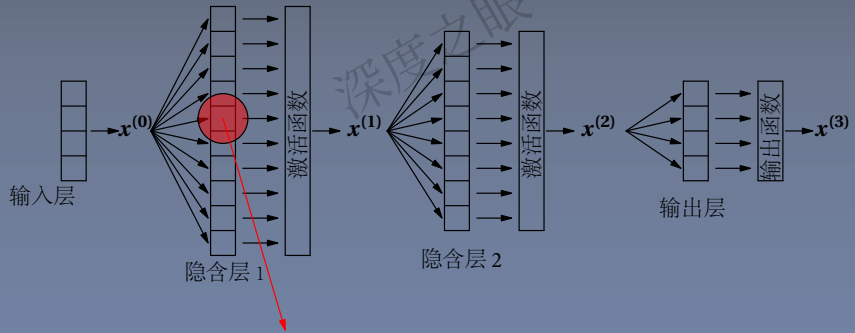
MLP Architecture & Activation Functions

MLP 架构

目标: 使用 MLP 近似一个函数 $\mathbf{f}: \mathbf{x} \in \mathbb{R}^d \mapsto \mathbf{y} \in \mathbb{R}^D$。

组成:

- **输入层**: 提供输入 $\mathbf{x} \in \mathbb{R}^d$
- **隐藏层**: 堆叠多个神经元层
- **输出层**: 输出预测 $\mathbf{y} \in \mathbb{R}^D$



$$x_i^{(l)} = \sigma(W_{ij}^{(l)} x_j^{(l-1)} + b_i^{(l)})$$

深度之眼
deepshare.net

MLP 运算

深度之眼 deepshare.net

运算公式:

$$\mathbf{x}^{(l)} = \sigma\left(\mathcal{A}^{(l)}(\mathbf{x}^{(l-1)})\right), \quad \mathcal{A}^{(l)}(\mathbf{x}^{(l-1)}) = \mathbf{W}^{(l)}\mathbf{x}^{(l-1)} + \mathbf{b}^{(l)}$$

$$\mathcal{F}(\mathbf{x}) = \mathcal{A}^{(L+1)} \circ \sigma \circ \mathcal{A}^{(L)} \cdots \sigma \circ \mathcal{A}^{(1)}(\mathbf{x})$$

参数:

$$\boldsymbol{\theta} = \{\mathbf{W}^{(l)}, \mathbf{b}^{(l)}\}_{l=1}^{L+1}$$

深度: $L+1$

深度之眼

激活函数概览

激活函数是 MLP 中的核心组件，帮助网络表示复杂的非线性关系。常见的激活函数包括:

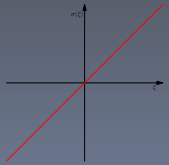


图: 线性

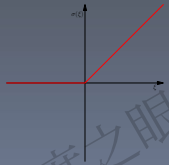


图: ReLU

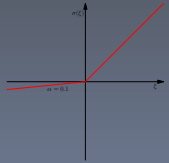


图: Leaky ReLU

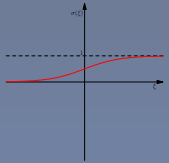


图: Sigmoid

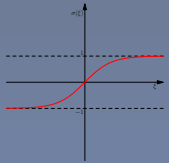


图: Tanh

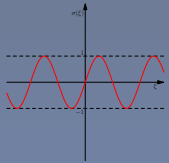


图: Sine

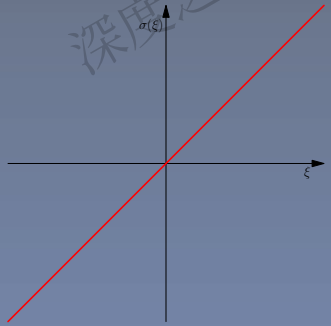
深度之眼
深度之眼
deepshare.net

线性激活函数

$$\sigma(\xi) = \xi$$

特点:

- 无穷光滑，但高阶导数为零
- 输出范围: $(-\infty, \infty)$
- 无法表示非线性函数，容易退化为线性变换



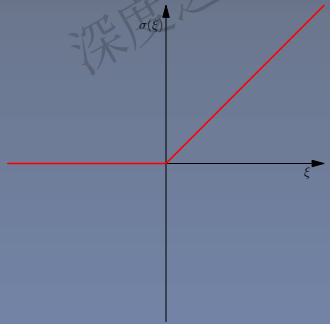
深度之眼
deepshare.net
深度之眼

ReLU 激活函数

$$\sigma(\xi) = \max\{0, \xi\}$$

特点:

- 分段线性，导数分段常数
- 输出范围: $[0, \infty)$
- 适用于大多数深度学习模型



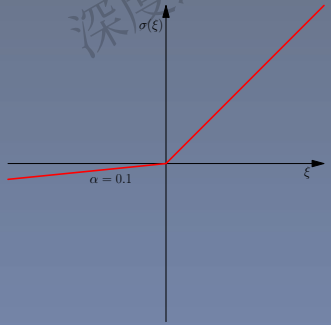
深度之眼
深度之眼
deepshare.net

Leaky ReLU 激活函数

$$\sigma(\xi;\alpha)=\begin{cases}\xi, & \xi\geq 0\\ \alpha\xi, & \xi<0\end{cases}$$

特点:

- 避免神经元“死亡”问题
- 输出范围: $(-\infty,\infty)$



深度之眼
深度之眼
deepshare.net

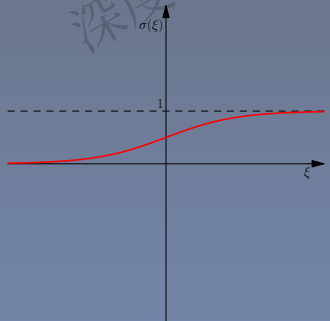
Sigmoid 激活函数

$$\sigma(\xi) = \frac{1}{1 + e^{-\xi}}$$

特点:

- 无穷光滑，值域 $(0, 1)$
- 导数在远离 $\xi = 0$ 时衰减
- 常用于二分类任务

深度之眼



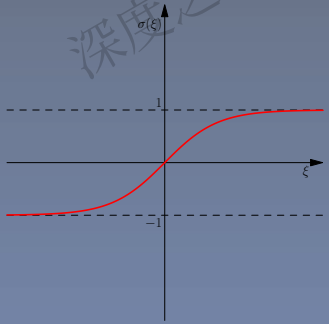
双曲正切 (Tanh)

$$\sigma(\xi) = \frac{e^{\xi} - e^{-\xi}}{e^{\xi} + e^{-\xi}}$$

特点:

- 无穷光滑，值域 $(-1, 1)$
- 中心对称，**常用于PINN**

深度之眼

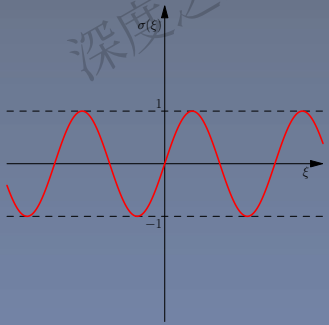


正弦激活函数 (Sine)

$$\sigma(\xi) = \sin(\xi)$$

特点:

- 无穷光滑，值域 $(-1, 1)$
- 导数不会衰减，**适用于物理建模/PINN**



深度之眼
deepshare.net
深度之眼

深度之眼
deepshare.net

损失函数与网络训练

Loss Functions & Training

损失函数的定义

深度之眼 deepshare.net

定义:

- 损失函数用于衡量模型预测 $\hat{\mathbf{y}}$ 与真实值 $\mathbf{y}$ 之间的误差。
- 目标是最小化损失函数，从而提高模型的预测准确度。

数学表达:

$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{N}\sum_{i=1}^{N} \ell(\hat{\mathbf{y}}_i, \mathbf{y}_i)$$

其中，$\ell(\hat{\mathbf{y}}_i, \mathbf{y}_i)$ 是单个样本的损失，N 为样本数，$\boldsymbol{\theta}$ 代表模型参数。

深度之眼

常见损失函数

深度之眼 deepshare.net

回归任务:

- **均方误差 (MSE)**:

$$\ell_{\text{MSE}}(\hat{\mathbf{y}}, \mathbf{y}) = \frac{1}{N}\sum_{i=1}^{N}(\hat{\mathbf{y}}_i - \mathbf{y}_i)^2$$

- **均绝对误差 (MAE)**:

$$\ell_{\text{MAE}}(\hat{\mathbf{y}}, \mathbf{y}) = \frac{1}{N}\sum_{i=1}^{N}|\hat{\mathbf{y}}_i - \mathbf{y}_i|$$

深度之眼

分类任务:

- **交叉熵损失 (Cross-Entropy Loss)**:

$$\ell_{\text{CE}}(\hat{\mathbf{y}}, \mathbf{y}) = -\sum_{i=1}^{C} y_i \log(\hat{y}_i)$$

其中，C 为类别数，y_i 是独热编码（one-hot encoding），$\hat{y}_i$ 是预测概率。

损失函数在优化中的作用

深度之眼 deepshare.net

梯度下降:

- 损失函数 $\mathcal{L}(\boldsymbol{\theta})$ 决定了优化的目标。
- 通过梯度下降优化：

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \frac{\partial \mathcal{L}}{\partial \boldsymbol{\theta}}$$

  其中 η 为学习率。

影响因素:

- 损失函数的选择影响收敛速度和最终精度。
- 不适当的损失函数可能导致梯度消失或梯度爆炸问题。

深度之眼

数据集定义

深度之眼
deepshare.net

假设数据集为：

$$\mathcal{S} = \{(\mathbf{x}_i, \mathbf{y}_i) : 1 \leq i \leq N\}$$

- 对应的目标函数为：$\mathbf{f} : \mathbf{x} \mapsto \mathbf{y}$
- 使用神经网络逼近该函数：

$$\mathcal{F}(\mathbf{x}; \boldsymbol{\theta}, \Theta)$$

- 其中：
  - $\boldsymbol{\theta}$: 网络参数
  - Θ: 网络超参数（深度 $L+1$、宽度 H、激活函数 σ 等）

深度之眼

关键步骤与数据集划分

深度之眼
deepshare.net

设计一个鲁棒的神经网络包括以下三个步骤：

1. **训练阶段：** 确定最佳网络参数 $\boldsymbol{\theta}$。
2. **验证阶段：** 确定最佳网络超参数 Θ。
3. **测试阶段：** 评估网络在未见数据上的表现。

深度之眼

数据集 $\mathcal{S}$ 的划分：

- **训练集：** N_{train} 样本（60%）
- **验证集：** N_{val} 样本（20%）
- **测试集：** N_{test} 样本（20%）

目的： 防止过拟合，提高网络的泛化能力。

训练阶段

深度之眼 deepshare.net

目标：最小化训练损失：

$$\boldsymbol{\theta}^* = \arg\min_{\boldsymbol{\theta}} \mathcal{L}_{\text{train}}(\boldsymbol{\theta})$$

其中：

$$\mathcal{L}_{\text{train}}(\boldsymbol{\theta}) = \frac{1}{N_{\text{train}}} \sum_{\substack{i=1 \\ (\mathbf{x}_i, \mathbf{y}_i) \in \mathcal{S}_{\text{train}}}}^{N_{\text{train}}} \|\mathbf{y}_i - \mathcal{F}(\mathbf{x}_i; \boldsymbol{\theta}, \Theta)\|^2$$

深度之眼

- 损失函数为均方误差（MSE）。
- $\boldsymbol{\theta}^*$：通过梯度下降法等优化算法获得的最佳参数。

验证阶段

深度之眼
deepshare.net

目标: 优化超参数 Θ：

$$\Theta^* = \arg\min_{\Theta} \mathcal{L}_{\text{val}}(\Theta)$$

其中：

$$\mathcal{L}_{\text{val}}(\Theta) = \frac{1}{N_{\text{val}}} \sum_{\substack{i=1 \\ (\mathbf{x}_i, \mathbf{y}_i) \in \mathcal{S}_{\text{val}}}}^{N_{\text{val}}} \|\mathbf{y}_i - \mathcal{F}(\mathbf{x}_i; \boldsymbol{\theta}^*, \Theta)\|^2$$

深度之眼

- 常用方法：网格搜索、随机搜索。
- 输出最佳超参数 Θ^*。

测试阶段

深度之眼
deepshare.net

目标: 测试网络在未见数据上的表现:

$$\mathcal{L}_{\text{test}} = \frac{1}{N_{\text{test}}} \sum_{\substack{i=1 \\ (\mathbf{x}_i, \mathbf{y}_i) \in \mathcal{S}_{\text{test}}}}^{N_{\text{test}}} \|\mathbf{y}_i - \mathcal{F}(\mathbf{x}_i; \boldsymbol{\theta}^*, \Theta^*)\|^2$$

深度之眼

- 该误差称为网络的泛化误差。
- 结果用于评估模型性能和鲁棒性。

深度之眼
deepshare.net

梯度下降与反向传播

Gradient Descent & Backpropagation

梯度的定义

深度之眼
deepshare.net

在多维空间中，**梯度**（Gradient）是一个向量，它描述了函数在某一点的变化率和变化方向。
假设我们有一个标量函数 $f(\boldsymbol{\theta})$，其中 $\boldsymbol{\theta} = (\theta_1, \theta_2, \ldots, \theta_n)$ 是一个 n 维向量。函数 f 的梯度是一个向量，表示损失函数在当前点 $\boldsymbol{\theta}$ 处对各个参数的偏导数：

$$\nabla f(\boldsymbol{\theta}) = \left(\frac{\partial f}{\partial \theta_1}, \frac{\partial f}{\partial \theta_2}, \ldots, \frac{\partial f}{\partial \theta_n} \right)$$

深度之眼

该梯度向量指向函数增长最快的方向，且梯度的大小表示该方向上函数的变化速度。
梯度的物理意义： 梯度是函数的方向导数，指示了在给定点附近，函数值的最大增加方向。

梯度的几何意义

深度之眼
deepshare.net

梯度不仅是数学工具，它有重要的几何意义。设 $f(\theta)$ 是一个标量函数，θ 是一个向量，$\nabla f(\theta)$ 是 f 在点 θ 的梯度。我们可以用以下几何图示来理解梯度：

- **梯度方向：** 梯度指向函数值增加最快的方向。
- **梯度大小：** 梯度的大小（模长）表示函数值增加的速率。如果梯度的大小较大，表示函数变化得较快；如果梯度的大小较小，表示函数变化较慢。

直观地，梯度就像是在山坡上的坡度，它告诉我们哪一个方向最陡，沿这个方向走，能够最快地上升（增加函数值）。

梯度在优化中的作用

深度之眼
deepshare.net

在优化问题中，我们希望通过调整参数 θ 来使得损失函数 $L(\theta)$ 最小化。为了做到这一点，梯度起到了非常重要的作用：

- **梯度上升：** 如果我们沿梯度方向更新参数，损失函数将会增加，因为梯度是损失函数变化最快的上升方向。
- **梯度下降：** 为了最小化损失函数，我们需要沿着梯度的反方向（负梯度方向）更新参数，这样损失函数会减少。

总结： 梯度的作用就是告诉我们在每个参数方向上，损失函数的变化率，帮助我们决定在参数空间中如何调整参数，以达到最小化损失的目标。

深度之眼

梯度下降基本思想

深度之眼 deepshare.net

目标： 优化损失函数 $\mathcal{L}_{\text{train}}(\theta)$，调整参数 θ。
梯度下降的目标是通过不断调整模型的参数，减少损失函数的值，从而使模型更好地拟合数据。
更新公式：

$$\theta^{(t+1)} = \theta^{(t)} - \eta \nabla_\theta \mathcal{L}_{\text{train}}(\theta^{(t)})$$

- η: 学习率，控制每次更新的步长大小。如果学习率过大，可能会跳过最优解；如果过小，收敛速度可能会变慢。
- $\nabla_\theta \mathcal{L}_{\text{train}}$: 损失函数的梯度，表示损失函数在当前参数点的变化率。它告诉我们在每个参数方向上，损失函数如何变化。
- **沿负梯度方向更新：** 为了最小化损失函数，我们沿着梯度的反方向（即负梯度方向）更新参数，从而减少损失值。

总结： 梯度下降通过计算损失函数关于参数的梯度，并在负梯度方向上更新参数，逐步逼近损失函数的最小值。

深度之眼

负梯度最小化推导

深度之眼
deepshare.net

假设我们有一个标量损失函数 $\mathcal{L}(\boldsymbol{\theta})$，我们希望最小化它。对损失函数 $\mathcal{L}(\boldsymbol{\theta})$ 进行泰勒展开，得到：

$$\mathcal{L}(\boldsymbol{\theta}) \approx \mathcal{L}(\boldsymbol{\theta}_0) + \nabla_{\boldsymbol{\theta}}\mathcal{L}(\boldsymbol{\theta}_0)^\top(\boldsymbol{\theta} - \boldsymbol{\theta}_0) + \frac{1}{2}(\boldsymbol{\theta} - \boldsymbol{\theta}_0)^\top H(\boldsymbol{\theta} - \boldsymbol{\theta}_0)$$

其中，H 是海森矩阵，表示二阶导数矩阵。
进行梯度下降更新：

$$\boldsymbol{\theta} = \boldsymbol{\theta}_0 - \eta\nabla_{\boldsymbol{\theta}}\mathcal{L}(\boldsymbol{\theta}_0)$$

代入泰勒展开式，得到：

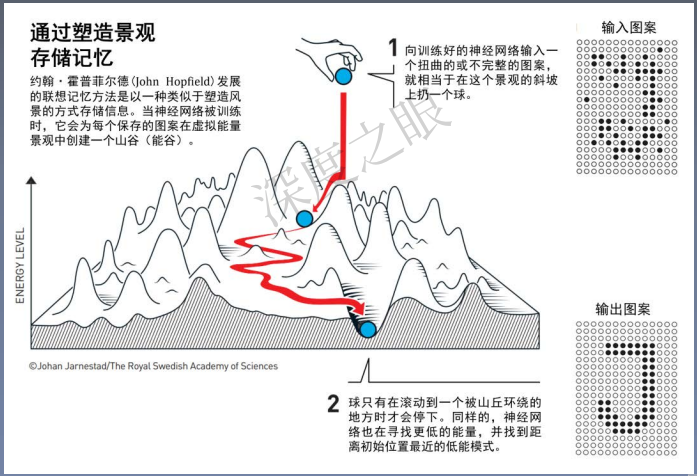
$$\mathcal{L}(\boldsymbol{\theta}) \approx \mathcal{L}(\boldsymbol{\theta}_0) - \eta\nabla_{\boldsymbol{\theta}}\mathcal{L}(\boldsymbol{\theta}_0)^\top\nabla_{\boldsymbol{\theta}}\mathcal{L}(\boldsymbol{\theta}_0) + O(\eta^2)$$

由此可见，随着步长 η 的增大，第二项（$-\eta\nabla_{\boldsymbol{\theta}}\mathcal{L}(\boldsymbol{\theta}_0)^\top\nabla_{\boldsymbol{\theta}}\mathcal{L}(\boldsymbol{\theta}_0)$）使得损失函数值减小。因此，沿着负梯度方向更新参数，能够使损失函数值减少。

深度之眼

梯度下降的几何解释

- 类比：在能量景观中，小球沿坡下降到最低点。
- 更新使损失函数值 $\mathcal{L}_{\text{train}}$ 逐步减少。



常见的梯度下降算法

深度之眼 deepshare.net

常见的梯度下降优化算法包括以下几种：

- **随机梯度下降 (SGD)**: 基本的梯度下降算法，计算每个小批量数据的梯度并更新参数。
- **带动量的梯度下降 (SGD with Momentum)**: 在更新参数时加入上一轮的梯度信息，帮助加速收敛。
- **自适应梯度算法 (Adagrad)**: 根据每个参数的历史梯度平方和，调整每个参数的学习率。
- **RMSProp**: 自适应学习率优化算法，基于移动平均的方式调整每个参数的学习率。
- **Adam**: 结合了动量法和RMSProp的优点，是目前应用最广泛的优化算法之一。

深度之眼

每种算法有其不同的特点和适用场景，选择合适的优化算法能够有效提高模型训练的效率和性能。

Adam

深度之眼
deepshare.net

Adam (Adaptive Moment Estimation) 是目前最广泛使用的优化算法，它结合了Momentum和RMSProp的优点，通过对梯度的一阶矩和二阶矩进行估计来更新参数。更新公式为：

$$\boldsymbol{m}^{(t+1)} = \beta_1 \boldsymbol{m}^{(t)} + (1-\beta_1)\nabla_\theta \mathcal{L}_{\text{train}}(\boldsymbol{\theta}^{(t)})$$

$$\boldsymbol{v}^{(t+1)} = \beta_2 \boldsymbol{v}^{(t)} + (1-\beta_2)(\nabla_\theta \mathcal{L}_{\text{train}}(\boldsymbol{\theta}^{(t)}))^2$$

$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \frac{\eta \boldsymbol{m}^{(t+1)}}{\sqrt{\boldsymbol{v}^{(t+1)}} + \epsilon}$$

深度之眼

其中，β_1 和 β_2 是控制一阶和二阶矩估计的衰减率。

- Adam 可以自适应调整每个参数的学习率，适用于大多数问题，尤其在神经网络训练中效果突出。
- 在PyTorch中，使用 ‘torch.optim.Adam‘ 来实现。

为什么需要反向传播？

深度之眼 deepshare.net

核心问题：如何优化参数θ使$\mathcal{F}(\mathbf{x})$逼近真实函数$\mathbf{f}(\mathbf{x})$？

- **损失函数**：$\mathcal{L}(\theta) = \frac{1}{2}\|\mathbf{y} - \mathcal{F}(\mathbf{x})\|^2$
- **优化目标**：$\theta^* = \arg\min_{\theta} \mathcal{L}(\theta)$
- **梯度下降**：$\mathbf{W}^{(l)} \leftarrow \mathbf{W}^{(l)} - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}}$
  $\mathbf{b}^{(l)} \leftarrow \mathbf{b}^{(l)} - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(l)}}$

深度之眼

关键挑战

如何高效计算数百万参数的梯度？$\rightarrow$ 反向传播算法

反向传播原理

深度之眼 deepshare.net

链式法则: 从输出层到输入层反向计算梯度

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}} = \underbrace{\frac{\partial \mathcal{L}}{\partial \mathbf{x}^{(L)}}}_{\text{顶层梯度}} \cdot \underbrace{\prod_{k=L}^{l+1} \frac{\partial \mathbf{x}^{(k)}}{\partial \mathbf{x}^{(k-1)}}}_{\text{路径累积}} \cdot \underbrace{\frac{\partial \mathbf{x}^{(l)}}{\partial \mathbf{W}^{(l)}}}_{\text{本地梯度}}$$

反向传播算法步骤:

1. **前向传播**：计算各层激活值$\mathbf{x}^{(l)}$
2. **反向计算**：从输出层开始逐层计算梯度
   - 输出层梯度：$\delta^{(L)} = \frac{\partial \mathcal{L}}{\partial \mathbf{x}^{(L)}}$
   - 隐藏层梯度：$\delta^{(l)} = (\mathbf{W}^{(l+1)T}\delta^{(l+1)}) \odot \sigma'(\mathcal{A}^{(l)})$
3. **参数更新**：

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}} = \delta^{(l)}(\mathbf{x}^{(l-1)})^T; \quad \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(l)}} = \delta^{(l)}$$

计算技巧: 复用前向传播结果，复杂度仅为$\mathcal{O}(N)$而非$\mathcal{O}(N^2)$

深度之眼

深度之眼
deepshare.net

Pytorch回归任务实例

Pytorch Regression Example

Pytorch 配置与资源

深度之眼 deepshare.net

Pytorch 配置与安装

- **Pytorch 基本配置:**
  配置指南，详细介绍 Pytorch 环境：
  https://blog.51cto.com/u16213384/11343925
- **Pytorch + Spyder 配置:**
  适合使用 Spyder 环境的教程：
  https://blog.panpili.com/2021/coding/ml/install-pytorch/

深度之眼

Kaggle 免费 GPU 资源： https://blog.51cto.com/u16099358/8789563

Pytorch 教学材料

- **Pytorch 教学教程:**
  从基础到高级的 Pytorch 教程代码仓库：
  https://github.com/yunjey/pytorch-tutorial

PyTorch 回归任务 - 拟合 $\sin(2\pi x)$

深度之眼 deepshare.net

目标：使用神经网络模型来拟合函数 $y = \sin(2\pi x)$，这是一个简单的回归问题。

步骤：

- 准备训练数据：生成一些样本点 $\{x_i, y_i\}$，其中 $x_i \in [0, 1]$，$y_i = \sin(2\pi x_i)$。
- 定义神经网络模型。
- 使用均方误差（MSE）损失函数来衡量预测值和真实值之间的差异。
- 使用优化算法（如Adam）来更新网络参数，最小化损失函数。

以下是实现的完整代码：

深度之眼

```python
import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split

# 生成训练数据
x_train = torch.linspace(0, 1, 100).reshape(-1, 1)
y_train = torch.sin(2 * np.pi * x_train)
```

PyTorch 回归任务 - 拟合 $\sin(2\pi x)$

深度之眼 deepshare.net

```
# 拆分为训练集、验证集和测试集
x_train, x_temp, y_train, y_temp = train_test_split(x, y, test_size
    =0.4, random_state=42)
x_val, x_test, y_val, y_test = train_test_split(x_temp, y_temp,
    test_size=0.5, random_state=42)

# 定义神经网络模型
class SimpleNN(nn.Module):
 def __init__(self):
 super(SimpleNN, self).__init__()
 self.hidden = nn.Linear(1, 64) # 隐藏层
 self.output = nn.Linear(64, 1) # 输出层

def forward(self, x):
 x = torch.relu(self.hidden(x)) # ReLU激活函数
 return self.output(x)
```

深度之眼

PyTorch 回归任务 - 拟合 $\sin(2\pi x)$

深度之眼
deepshare.net

```python
# 初始化模型、损失函数和优化器
model = SimpleNN()
criterion = nn.MSELoss() # 均方误差损失函数
optimizer = optim.Adam(model.parameters(), lr=0.001) # Adam优化器

# 训练模型
num_epochs = 5000
train_losses = []
val_losses = []

for epoch in range(num_epochs):
 model.train()
 optimizer.zero_grad() # 清除梯度
 output_train = model(x_train) # 前向传播
 loss_train = criterion(output_train, y_train) # 计算训练集损失
 loss_train.backward() # 反向传播
 optimizer.step() # 更新参数
```

深度之眼

PyTorch 回归任务 - 拟合 $\sin(2\pi x)$

深度之眼 deepshare.net

```
for epoch in range(num_epochs):
 ......

 # 计算验证集损失
 model.eval()
 with torch.no_grad():
 output_val = model(x_val)
 loss_val = criterion(output_val, y_val)

 # 保存训练集和验证集的损失
 train_losses.append(loss_train.item())
 val_losses.append(loss_val.item())

 # 每500步输出训练和验证集的损失
 if (epoch + 1) % 500 == 0:
  print(f'Epoch [{epoch+1}/{num_epochs}], Train Loss: {loss_train.
      item():.4f}, Val Loss: {loss_val.item():.4f}')
```

深度之眼

PyTorch 回归任务 - 拟合 $\sin(2\pi x)$

深度之眼 deepshare.net

```
# 绘制训练集和验证集的损失曲线
plt.plot(train_losses, label='Training Loss')
plt.plot(val_losses, label='Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.title('Loss Curves')
plt.show()

# 测试模型
model.eval()
with torch.no_grad():
predicted_test = model(x_test).detach()
```

深度之眼

PyTorch 回归任务 - 拟合 $\sin(2\pi x)$

深度之眼 deepshare.net

```
# 绘制结果
plt.plot(x.numpy(), y.numpy(), label='True value')
plt.scatter(x_test.numpy(), y_test.numpy(), label='Test value',
    color='blue', s=20)
plt.scatter(x_test.numpy(), predicted_test.numpy(), label='
    Predicted value', color='red', s=20)
plt.legend()
plt.title('Test Set Prediction')
plt.show()
```

深度之眼

训练过程：

- 该代码将训练神经网络模型，以便拟合 $\sin(2\pi x)$ 函数。
- 使用了Adam优化器和均方误差（MSE）损失函数来进行训练。
- 每500次迭代输出一次损失值，以跟踪训练进度。

回归任务结果

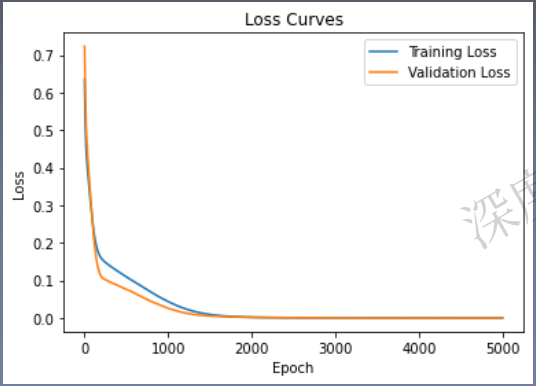


图: 损失函数

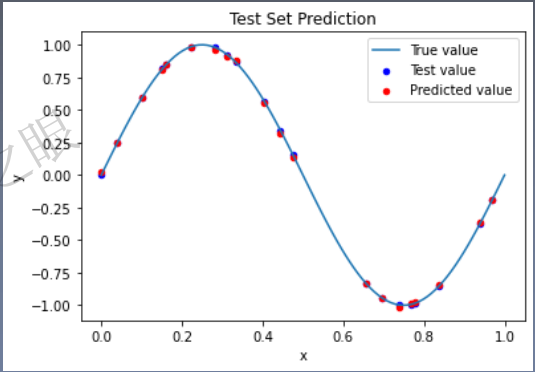


图: 预测结果

深度之眼
deepshare.net
深度之眼

深度之眼

deepshare.net

公众号